"Hit my target" planner: `engine/target.py`

Answers: "Cut the bill by X%. What is the least risky set of changes that gets there?" The user types a percentage; the planner returns a plan, or an honest "this cannot be done safely".

The same algorithm is mirrored in the website code (`lib/engine.ts`, `planForTarget`) so the live page recomputes plans in the browser. The two versions must give the same answers.

1. What it does

Inputs: the list of recommendation cards, the current monthly bill, the target percentage, and an optional set of cards the user has excluded. **Output:** a plan with the chosen changes (each with action, risk, risk score, saving and phase), the achieved saving and percentage, the saving-weighted plan risk, the cost of the required upsizes, a message, and three tiers (the most each risk level can save).

2. Which cards can be picked

- **Eligible actions:** resize, schedule, stop, delete_volume, release_ip, and only if the saving is positive, the card is not excluded and its risk is within the allowed level.
- **Never picked:** keep, review and investigate cards. These need a human decision, so the planner never counts on them.
- **Required (always in the plan):** upsizes that are accepted by default. They fix VMs that are already too small, so they are included automatically, and their extra cost is subtracted from the saving.

3. Options per resource

Each resource offers a small menu, and the plan picks at most one option per resource:

1. do nothing
2. each eligible card alone (resize, or schedule, or stop, and so on)
3. **resize + schedule together**, when both exist: savings are added, the risk is the higher of the two, and the risk scores are added

A schedule chosen **without** its resize runs on the larger current size, so it saves more. The planner uses `saving_alone` (current price × stopped fraction) for that case.

4. Algorithm: exact search, not greedy

A greedy rule ("take the biggest saving first") can take extra risk when a smaller combination would have worked. The planner uses a **multiple-choice knapsack** (dynamic programming):

1. Target in rupees = bill × percentage.
2. Needed saving from optional changes = target + cost of required upsizes.
3. Savings are counted in **₹100 steps**. `best[s]` holds the lowest-risk way found so far to reach `s` steps using the resources seen up to now.
4. For each resource, for each existing state and each option, compute the new state `s +`

`saving ÷ 100` (capped), and keep the option if its **risk key** is lower than what that state already holds.

5. After the last resource, take every state whose real rupee saving reaches the need, and pick the one with the lowest risk key, then the lowest real saving (least overshoot).
6. Walk backwards through the stored choices to list which option each resource got (a −1 means the resource is left alone).

Risk key (compared in this order, smaller is better; added up across the plan):

1. number of HIGH-risk changes
2. number of MEDIUM-risk changes
3. total risk score (×1000, rounded)
4. number of cards used

Then least overshoot as the final tie-break. So the plan adds MEDIUM or HIGH changes only when LOW ones cannot reach the target.

5. If the target cannot be reached

The planner returns the plan with the most saving the rules allow (preferring the lower-risk one on ties) and a message such as: "A 25% cut cannot be reached safely. Every safe change together saves ₹X (Y%)". VMs whose peaks would not fit a smaller size are never touched.

6. Phases (rollout order)

Phase	What
1 (week 1)	delete_volume, release_ip, stop, required upsizes: nothing running is disturbed
2	other changes on dev and staging
3	other changes on production

Items are sorted by phase, then risk, then saving.

7. Tiers and plan risk

- **Tiers:** for LOW, then up to MEDIUM, then up to HIGH, the sum over resources of the best option's saving, minus the upsize cost, shown as ₹ and % of the bill. They tell the user how far each risk level can go.
- **Plan risk:** the average risk score of the chosen changes, weighted by their savings.
- **Message:** "Reached with low-risk changes only", "Low-risk changes alone top out at X%, so the plan adds N medium-risk change(s)", or "Reached only with N high-risk change(s)".

8. Sample output (my run, seed 42)

Current bill ₹9,30,948 per month. Tiers: LOW 13.43%, up to MEDIUM 20.93%, up to HIGH 21.58%. A 15% target is not reachable with low-risk changes alone, so the plan adds 3 medium-risk changes.

9. Parameters

`STEP_INR` = 100 (the saving granularity), `ELIGIBLE` actions set, `PHASE` map. Risk scores and labels come from `recommend.py`; the planner does not change them.

10. Assumptions

- Savings from different resources add up (resize + schedule on the same VM is the one netted pair).
- The risk score from `recommend.py` is a fair measure of how risky a change is.
- A change takes full effect for the month.

11. Limitations

1. **"Exact" up to ₹100 rounding.** Each change's saving is rounded down to a whole ₹100 step while searching, so the result can be off by a small amount around the target boundary. The final check uses real rupees.
2. **Risk is a score, not a probability.** The ordering depends on how `recommend.py` assigns risk, so any bias there carries over.
3. **No dependencies between VMs.** Two services that depend on each other are treated independently.
4. **One option per resource.** Each resource gets a single card or the resize + schedule pair; no other pairings are considered.
5. **Two implementations.** The Python and TypeScript versions must be kept in sync by hand; the repo has a test file for the planner (`test_target.py`, 8 tests), but check it covers your own changes.